Residual Networks

Saturday, May 23, 2026 10:58 AM

- We saw last chapter that image classification performance improved as the depth of the network increased.
  - ↳ AlexNet (8 hidden layers) → VGG (19 hidden layers)
- This trend doesn't hold!
  - ↳ Performance decreases as more layers are added.
- This chapter introduces residual blocks!
  - ↳ Each network component computes an additive change to the current representation instead of transforming it directly.
  - ↳ Allows deeper networks to be trained, but causes an exponential increase in activation magnitudes at initialization.
  - ↳ Residual blocks use batch normalization to compensate for it; rescales the activations at each layer.
- Residual blocks w/ Batch normalization allow much deeper networks to be trained!
  - ↳ Also improves performance across a variety of tasks.

11.1: Sequential Processing:

- Every network seen so far processes the data sequentially.
  - ↳ Each layer takes the previous layer's output, & passes the result to the next:

$$h_1 = f_1[x, \phi_1]$$
$$h_2 = f_2[h_1, \phi_2]$$
$$h_3 = f_3[h_2, \phi_3]$$
$$y = f_4[h_3, \phi_4]$$

(x → Input; h_1, h_2, h_3 → Intermediate hidden layers; y → Output)

$f_K[\cdot, \phi_K]$ does the processing.

- In a standard NN, each layer is a linear transformation passed to the activation function, where ϕ_K are the parameters comprising the weights & biases of the transformation.



Figure 11.1 Sequential processing. Standard neural networks pass the output of each layer directly into the next layer.

- In a CNN, each layer consists of a set of convolutions followed by an activation function. The parameters comprise the kernels & biases.
  - ↳ The processing is still sequential. Can be thought of as a series of nested functions:

$$y = f_4[f_3[f_2[f_1[x, \phi_1], \phi_2], \phi_3], \phi_4]$$

Limitations of Sequential Processing:

- As we increase layers, performance improves, then decreases!
- Surprising, as models generally perform better when more capacity is added

- As we increase layers, performance improves, then decreases!
- Surprising, as models generally perform better when more capacity is added.
- The decrease in performance happens in both the training & testing set.
  - Implies the problem is training deeper networks rather than the inability of deeper networks to generalize.
  - This phenomena is not completely understood!

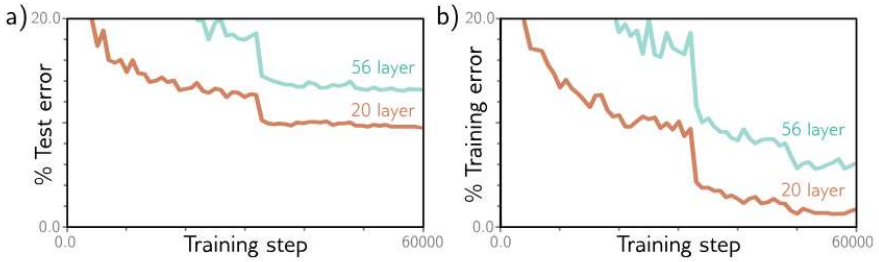


Figure 11.2 Decrease in performance when adding more convolutional layers. a) A 20-layer convolutional network outperforms a 56-layer neural network for image classification on the test set of the CIFAR-10 dataset (Krizhevsky & Hinton, 2009). b) This is also true for the training set, which suggests that the problem relates to training the original network rather than a failure to generalize to new data. Adapted from He et al. (2016a).

- Conjecture: Right after initialization, the loss gradients change unpredictably when modifying parameters in early network layers.

  With appropriate initialization of weights, the gradient of the loss will be reasonable [no vanishing/exploding gradients]. But the derivative assumes an infinitesimal change in the parameter, whereas optimization algorithms use a finite step size. Any reasonable choice of step size may move to a place with a completely different & unrelated gradient. The loss surface looks like a massive range of tiny mountains, rather than a smooth structure that's easy to descend. So the optimizer doesn't make progress in the way it does when the loss gradient changes more slowly!

- The conjecture is supported by empirical observations.
  - For a shallow network, the gradient of the output w/ respect to the input changes slowly as we change the input.
  - For a deep network, a tiny change in the input results in a completely different gradient.
  - This is captured by the autocorrelation function of the gradient.
  - Nearby gradients are correlated for shallow networks, but this correlation quickly drops to 0 for deep networks.
  - called the "Shattered gradients" phenomenon.

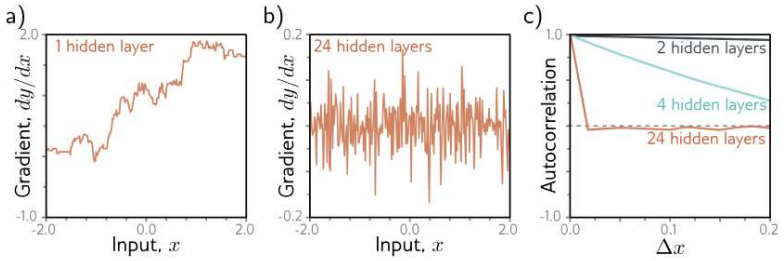


Figure 11.3 Shattered gradients. a) Consider a shallow network with 200 hidden units and Glorot initialization (He initialization without the factor of two) for both the weights and biases. The gradient $\partial y/\partial x$ of the scalar network output y with respect to the scalar input x changes relatively slowly as we change the input x. b) For a deep network with 24 layers and 200 hidden units per layer, this gradient changes very quickly and unpredictably. c) The autocorrelation function

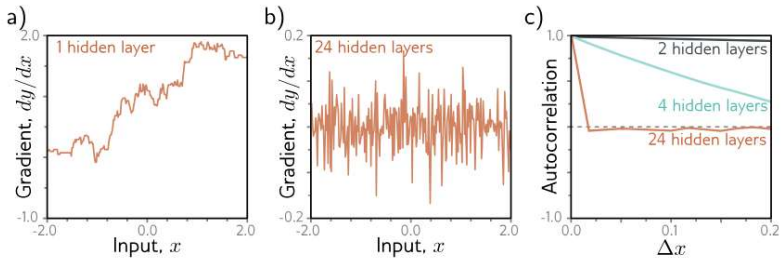


Figure 11.3 Shattered gradients. a) Consider a shallow network with 200 hidden units and Glorot initialization (He initialization without the factor of two) for both the weights and biases. The gradient $\partial y/\partial x$ of the scalar network output y with respect to the scalar input x changes relatively slowly as we change the input x. b) For a deep network with 24 layers and 200 hidden units per layer, this gradient changes very quickly and unpredictably. c) The autocorrelation function of the gradient shows that nearby gradients become unrelated (have autocorrelation close to zero) for deep networks. This *shattered gradients* phenomenon may explain why it is hard to train deep networks. Gradient descent algorithms rely on the loss surface being relatively smooth, so the gradients should be related before and after each update step. Adapted from Balduzzi et al. (2017).

- Shattered gradients arise because changes in early network layers modify the output in an increasingly complex way as networks get deeper.
  - ↳ The derivative of the output y w/ respect to the first layer in the network above is:

$$\frac{\partial y}{\partial f_1} = \frac{\partial f_2}{\partial f_1}\frac{\partial f_3}{\partial f_2}\frac{\partial f_4}{\partial f_3}$$

  - * When we change the parameters that determine f_1, all of the derivatives in this sequence are evaluated at slightly different locations, since f_2, f_3 & f_4 are themselves computed from f_1.
  - * So the updated gradient at each training example may become completely different. And the loss function becomes badly behaved.

11.2: Residual Connections & Residual Blocks:

- Residual or skip connections are branches in the computational path where the input to each network layer $f[\cdot]$ is added back to the output.
  - ↳ Defined as:

$$h_1 = x + f_1[x, \phi_1]$$
$$h_2 = h_1 + f_2[h_1, \phi_2]$$
$$h_3 = h_2 + f_3[h_2, \phi_3]$$
$$y = h_3 + f_4[h_3, \phi_4]$$

  - * The residual connection.
- Each function f_k learns an additive change to the current representation.
  - ↳ Their outputs must be the same size as their inputs.
- Each additive combination of the input & the processed output is known as the residual block / layer.
- Can be written as a single function by substituting the expressions for the intermediate quantities h_k:

$$y = x + f_1[x]$$

- can be written as a single function by substituting the expressions for the intermediate quantities h_k:

$$y = x + f_1[x] + f_2[x + f_1[x]] + f_3[x + f_1[x] + f_2[x + f_1[x]]] + f_4[x + f_1[x] + f_2[x + f_1[x]] + f_3[x + f_1[x] + f_2[x + f_1[x]]]]$$

*Parameters ϕ_k omitted for clarity

- We can think of the equation as "unraveling" the network.
  - The final network output is the sum of the input + 4 smaller networks, corresponding to each line of the equation.

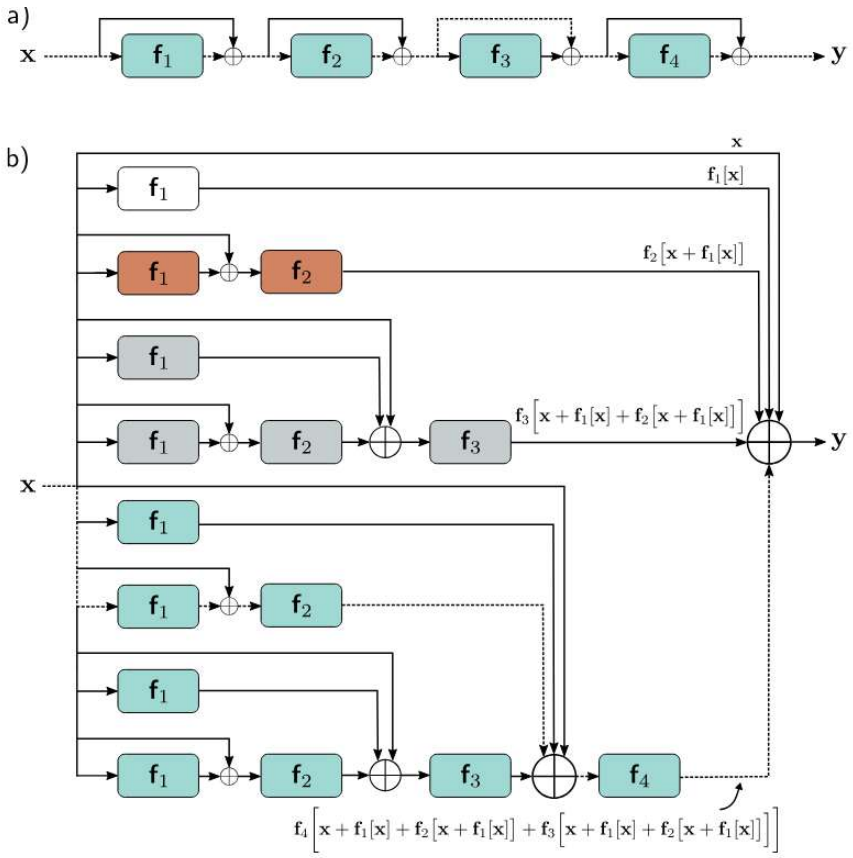


Figure 11.4 Residual connections. a) The output of each function $\mathbf{f}_k[\mathbf{x}, \phi_k]$ is added back to its input, which is passed via a parallel computational path called a residual or skip connection. Hence, the function computes an additive change to the representation. b) Upon expanding (unraveling) the network equations, we find that the output is the sum of the input plus four smaller networks (depicted in white, orange, gray, and cyan, respectively, and corresponding to terms in equation 11.5); we can think of this as an ensemble of networks. Moreover, the output from the cyan network is itself a transformation $\mathbf{f}_4[\bullet, \phi_4]$ of another ensemble, and so on. Alternatively, we can consider the network as a combination of 16 different paths through the computational graph. One example is the dashed path from input $\mathbf{x}$ to output $\mathbf{y}$, which is the same in panels (a) and (b).

- One interpretation is that residual connections turn the original network into an ensemble of these smaller networks whose outputs are summed to compute the result.
- An alternate view is that it creates 16 paths w/ differing numbers of transformations between input & output.
  - For example $f_1[x]$ occurs in 8 of the 16 paths. An analogous derivative of $\frac{\partial y}{\partial f_i}$ would be:

$$\frac{\partial y}{\partial f_1} = I + \frac{\partial f_2}{\partial} + \left(\frac{\partial f_2}{\partial} + \frac{\partial}{\partial} \frac{\partial}{\partial}\right) \left(\frac{\partial}{\partial} \cdot \frac{\partial}{\partial} \frac{\partial f_3}{\partial} \cdot \frac{\partial}{\partial} \frac{\partial f_3}{\partial} \frac{\partial}{\partial}\right)$$

↳ For example $f_1[x]$ occurs in 8 of the 16 paths. An analogous derivative of $\frac{\partial y}{\partial f_1}$ would be:

$$\frac{\partial y}{\partial f_1} = \underline{I} + \frac{\partial f_2}{\partial f_1} + \left(\frac{\partial f_3}{\partial f_1} + \frac{\partial f_2}{\partial f_1}\frac{\partial f_3}{\partial f_2}\right) + \left(\frac{\partial f_4}{\partial f_1} + \frac{\partial f_2}{\partial f_1}\frac{\partial f_4}{\partial f_2} + \frac{\partial f_3}{\partial f_1}\frac{\partial f_4}{\partial f_3} + \frac{\partial f_2}{\partial f_1}\frac{\partial f_3}{\partial f_2}\frac{\partial f_4}{\partial f_3}\right)$$

↳ 1 term for each of the 8 paths.

* The identity term shows that changes in the parameters ϕ_1 in the first layer $f_1[x, \phi_1]$ contribute directly to changes in the network output y.

* They also contribute indirectly through the other chains of derivatives of varying lengths.

- In general, gradients through shorter paths will be better behaved.

↳ Since both the identity term + various short chains of derivatives will contribute to the derivative for each layer residual networks suffer less from shattered gradients.

Order of operations in Residual Blocks:

- So far we implied that the additive functions $f[x]$ could be any valid network layer [eg. FC or convolutional]
- Technically true, but the order of operations in these functions is important.
- They must contain a nonlinear activation function like ReLU, or the entire function will be linear.

↳ In a typical network layer ReLU is at the end, so the output is non-negative. If we use this convention, then each residual block can only increase the output values.

↳ So its typical to change the order of operations so that the activation function is applied 1st, then the linear transformation.

↳ Sometimes there can be several layers of processing within the residual block, but these usually end w/ a linear transformation.

↳ When we start these blocks w/ ReLU, they will do nothing if the initial network input is negative, since ReLU clips the entire signal to 0. So its typical to start the network w/ a linear transformation rather than a residual block.

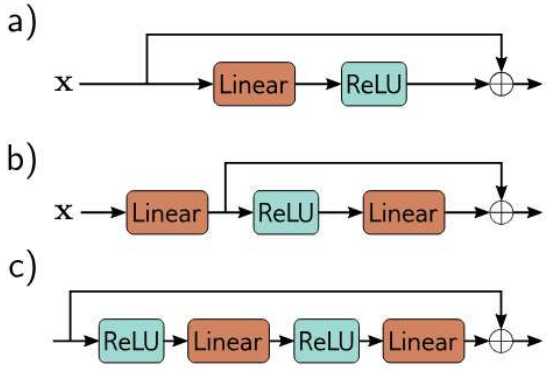


Figure 11.5 Order of operations in residual blocks. a) The usual order of linear transformation or convolution followed by a ReLU nonlinearity means that each residual block can only add non-negative quantities. b) With the reverse order, both positive and negative quantities can be added. However, we must add a linear transformation at the start of the network in case the input is all negative. c) In practice, it's common for a residual block to contain several network layers.

Deeper Networks w/ Residual Connections:

- Adding residual connections roughly doubles the depth of a network that can be practically trained before performance degrades.
- But we would like to increase the depth further.
- To understand why residual connections don't allow us to increase the depth arbitrarily, we consider how the variance of the activations changes during the forward pass & how the gradient magnitudes change during the backward pass.

Variance of the activations changes during the forward pass ; how the gradient magnitudes change during the backward pass.

11.3: Exploding Gradients in Residual Networks:

- w/o careful initialization, the magnitudes of the intermediate values can increase/decrease exponentially.
  - ↳ causes the gradients to explode/vanish during the backwards pass.

- So we initialize the parameters so that the expected variance of the activations [in the forward pass] ; gradients [in the backward pass] remains the same in between the layers.
  - ↳ He initialization does this for ReLu by initializing biases $\beta = 0$; choosing normally distributed weights Ω with mean 0 ; variance $2/D_h$, where D_h is the number of hidden units in the previous layer.
- In a residual network, its different!
  - ↳ Dont need to worry about intermediate values/gradients vanishing w/ network depth, since there is a path whereby each layer directly contributes to the network output.
  - ↳ But even if we use the initialization within the residual block, the values in the forward pass increase exponentially as we move through the network
  - ↳ This is because we add the result of the processing in the residual block back to the input. Each branch has some uncorrelated variability. So the overall variance increases when we recombine them
  - ↳ w/ ReLU activations ; He initialization, the expected variance is unchanged by the processing in each block. So when we recombine w/ the input, the variance doubles. Grows exponentially w/ the number of residual blocks.
- This limits the possible network depth before floating point precision is exceeded in the forward pass.
  - ↳ A similar argument applies to gradients in the backwards pass,
- So residual networks still suffer from unstable forward propagation ; exploding/vanishing gradients, even w/ the initialization.
- A couple solutions:
  1) Use He initialization, then multiply the combined output of the residual block by $1/\sqrt{2}$ to compensate for the doubling.
  2) Use Batch normalization

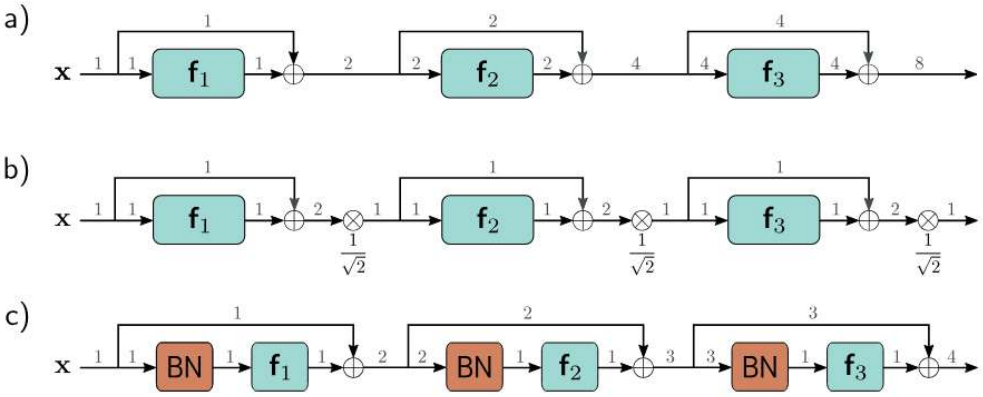


Figure 11.6 Variance in residual networks. a) He initialization ensures that the expected variance remains unchanged after a linear plus ReLU layer $\mathbf{f}_k$. Unfortunately, in residual networks, the input of each block is added back to the output, so the variance doubles at each layer (gray numbers indicate variance) and grows exponentially. b) One approach would be to rescale the signal by $1/\sqrt{2}$ between each residual block. c) A second method uses batch normalization (BN) as the first step in the residual block and initializes the associated offset δ to zero and scale γ to one. This transforms the input to each layer to have unit variance, and with He initialization, the output variance will also be one. Now the variance increases linearly with the number of residual blocks. A side-effect is that, at initialization, later network layers are dominated by the residual connection and are hence close to computing the identity.

11.4: Batch Normalization:

- Batch normalization shifts & rescales each activation h so that its mean & variance across the batch $\mathcal{B}$ becomes values that are learned during training.

- Steps:

1) The empirical mean m_h & standard deviation s_h are computed:

$$m_h = \frac{1}{|\mathcal{B}|}\sum_{i \in \mathcal{B}} h_i$$

$$s_h = \sqrt{\frac{1}{|\mathcal{B}|}\sum_{i \in \mathcal{B}} (h_i - m_h)^2}$$

All quantities are scalars.

2) Use these statistics to standardize the batch activations to have mean 0 & unit variance.

$$h_i \leftarrow \frac{h_i - m_h}{s_h + \epsilon} \quad \forall i \in \mathcal{B}$$

ϵ: A small number that prevents division by 0 if h_i is the same for every member of the batch & $s_h = 0$.

3) The normalized variable is scaled by γ & shifted by δ:

$$h_i \leftarrow \gamma h_i + \delta \quad \forall i \in \mathcal{B}$$

- After this, the activations have mean β & standard deviation γ across all members of the batch. Both β & γ are learned during training.

- Batch norm is applied independently to each hidden unit
  - In a standard NN w/ K layers, each w/ D hidden units, there are KD learned offsets & KD learned means scales.
  - In a CNN, the normalizing statistics are computed over both the batch & the spatial position. A CNN w/ K layers, each w/ C channels would have KC offsets & KC scales.

- At test time, we don't have a batch to compute the statistics.
  - So we calculate M_h & S_h are calculated across the whole training dataset, & frozen in the final network.

Costs & Benefits of Batch Normalization:

- Batch norm makes the network invariant to rescaling the weights & biases that contribute to each activation.
  - If they double, the activations double, the estimated standard deviation doubles, & the normalization compensates these changes.
  - Happens independently in each hidden unit.
  - There will be a large family of weights & biases that all produce the same effect. → Causes redundancy in the weights & biases!

- Also adds 2 parameters at every hidden unit, so it makes the model larger. → Adds extra parameters to compensate for the redundancy.

- Inefficient, but provides several benefits:

1) Stable Forward Propagation:

If we initialized the offsets β to 0 & the scales γ to 1, each output activation will have unit variance.

- In a regular network, it ensures the variance is stable during forward propagation.
- In a residual network, the variance must still increase as we add a new source of variation to the input at each layer. But it will increase linearly w/ each residual block. The Kth layer adds unit variance to the existing variance of K.
- At initialization, this has the side effect that later layers make a smaller change to the overall variation than earlier ones. The network is effectively less deep at the start of training since later layers are close to computing the identity. As training goes on, the network can increase the scales γ in later layers & control its effective depth.

2) Higher Learning Rates:

- Batch norm makes the loss surface & its gradient change more smoothly.
- We can use higher learning rates since the surface is more predictable.
- Higher learning rates improve test performance.

3) Regularization:

- Noise in the training process can improve generalization.

JJ Regularization:

- Noise in the training process can improve generalization.
- Batch Norm injects noise because the normalization depends on the batch statistics.
- So activations for a given training example are normalized by amounts that depend on other members of the batch which is different each iteration.

11.5: Common Residual Architectures:

ResNet:

- Residual blocks were first used in CNNs for image classification.
  - ↳ Known as residual networks/ ResNet.
  - ↳ Each residual block contains a batch norm, ReLU, & a conv layer. Followed by the same sequence again before being added back to the input.
- For very deep networks, the number of parameters may become undesirably large.
  - ↳ Bottleneck residual networks are more efficient using 3 convolutions.
  - ↳ The 1st was a 1x1 kernel & reduces the # of channels.
  - ↳ The 2nd is a normal 3x3 kernel.
  - ↳ The 3rd is a 1x1 kernel to increase the # of channels back to the original amount.

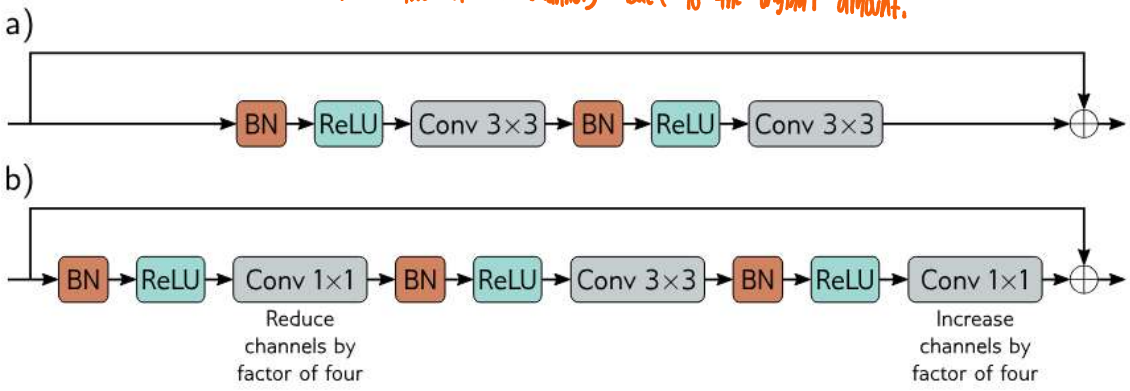
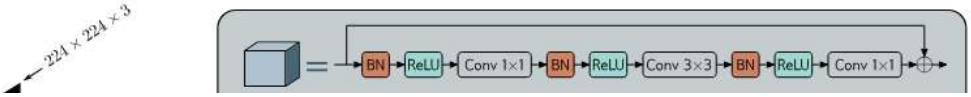


Figure 11.7 ResNet blocks. a) A standard block in the ResNet architecture contains a batch normalization operation, followed by an activation function, and a 3×3 convolutional layer. Then, this sequence is repeated. b). A bottleneck ResNet block still integrates information over a 3×3 region but uses fewer parameters. It contains three convolutions. The first 1×1 convolution reduces the number of channels. The second 3×3 convolution is applied to the smaller representation. A final 1×1 convolution increases the number of channels again so that it can be added back to the input.

- ResNet-200 achieved 4.8% error rate (top 5) & 20.1% error rate (top 1).
  - ↳ Better than AlexNet & VGG
  - ↳ One of the first models to exceed human performance.



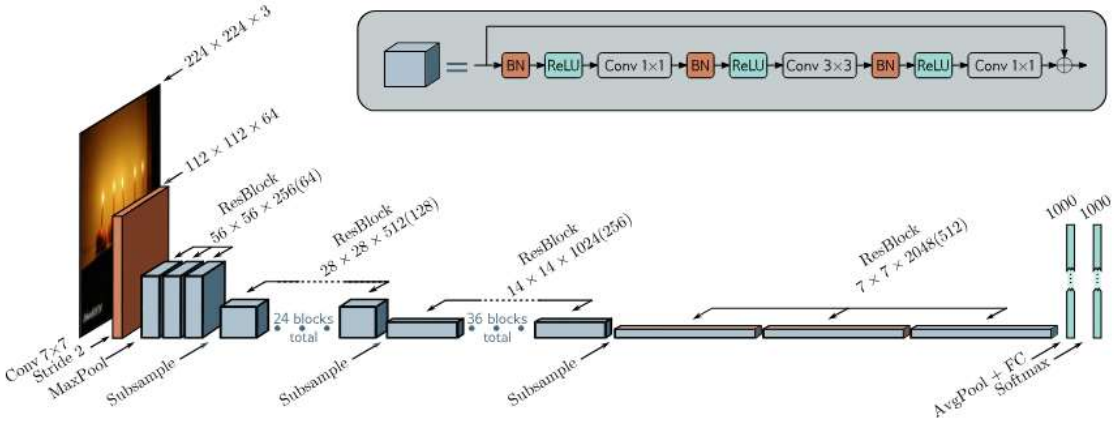


Figure 11.8 ResNet-200 model. A standard 7×7 convolutional layer with stride two is applied, followed by a MaxPool operation. A series of bottleneck residual blocks follow (number in brackets is channels after first 1×1 convolution), with periodic downsampling and accompanying increases in the number of channels. The network concludes with average pooling across all spatial positions and a fully connected layer that maps to pre-softmax activations.

Dense Net:

- Residual blocks receive the output from the previous layer, modify it by passing it through some network layers & add it back to the original input.
  - ↳ Alternative: Concatenate the modified & original signals.
  - ↳ This increases the representation size [channels for a CNN] but an optimal subsequent linear transformation can map back to the original size.
  - ↳ This allows the model to add the representations together, take a weighted sum, or combine them in a more complex way.
  - ↳ DenseNet uses concatination so that the input to a layer comprises the concatenated outputs of all previous layers.

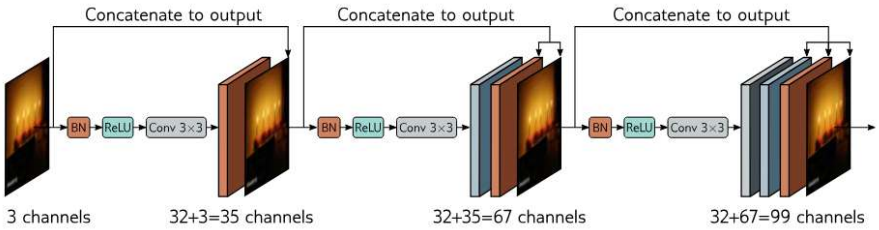


Figure 11.9 DenseNet. This architecture uses residual connections to concatenate the outputs of earlier layers to later ones. Here, the three-channel input image is processed to form a 32-channel representation. The input image is concatenated to this to give a total of 35 channels. This combined representation is processed to create another 32-channel representation, and both earlier representations are concatenated to this to create a total of 67 channels and so on.

- This concatination means there is a direct contribution from earlier layers to the output, so the loss behaves reasonably.
- In practice, this can only be sustained for a few layers.

↳ The # of channels [; thus parameters] become increasingly large.

↳ Can be alleviated by applying a 1x1 convolution to reduce the # of channels before the next 3x3 convolution is applied.

↳ In CNNs the input is periodically downsampled. So concatination makes no sense, since the representations have different sample sizes. So the chain of concatenation is broken at this point, ; a smaller representation starts a new chain.

↳ Also another bottleneck 1x1 convolution can be applied when the downsampling occurs to control the representation size further.

- Dense Nets perform competitively with ResNet models on image classification. It can perform better for a comparible parameter count.
  ↳ Presumably because it can reuse processing from earlier layers more flexibly.

U-Nets ; Hourglass Networks:

- Last chapter we covered a semantic segmentation network with an encoder-decoder / hourglass structure.
- The encoder repeatedly downsamples the image until the receptive fields are large ; information is integrated from across the image.
- The decoder then upsamples it back to the size of the original image.
- The final output is a probability over possible object classes at each pixel.
- One drawback is that the bottleneck must "remember" the high resolution details to make the final result accurate.
  ↳ Unnecessary if residual connections transfer the representations from the encoder to their partner in the decoder.
- The U-Net is an encoder-decoder architecture where the earlier representations are concatenated to the later ones.
  ↳ The original implementation used "valid" convolutions, so the spatial size decreases by 2 pixels each time a 3x3 convolutional layer is applied.

↳ Means the upsampled version is smaller than its counterpart in the encoder, which must be cropped before concatenation.

↳ Subsequent implementations have used zero-padding where this cropping is unnecessary.

↳ The U-Net is completely convolutional, so after training it can be ran on an image of any size.

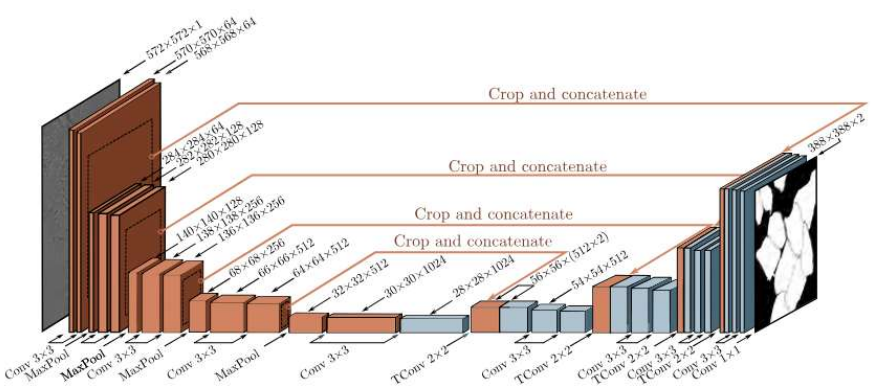


Figure 11.10 U-Net for segmenting HeLa cells. The U-Net has an encoder-decoder structure, in which the representation is downsampled (orange blocks) and then re-upsampled (blue blocks). The encoder uses regular convolutions, and the decoder uses transposed convolutions. Residual connections append the last representation at each scale in the encoder to the first representation at the same scale in the decoder (orange arrows). The original U-Net used "valid" convolutions, so the size decreased slightly with each layer, even without downsampling. Hence, the representations from the encoder were cropped (dashed squares) before appending to the decoder. Adapted from Ronneberger et al. (2015).

- Hourglass Networks are similar but apply further convolutional layers in the skip connections & add the result back to the decoder rather than concatenating it.
- A series of these models form a stacked hourglass network that alternates between considering the image at local & global levels.
  - ↳ These networks are used for pose estimation.
  - ↳ Trained to predict one "heatmap" for each joint, & the estimated position is the maximum of each heatmap.

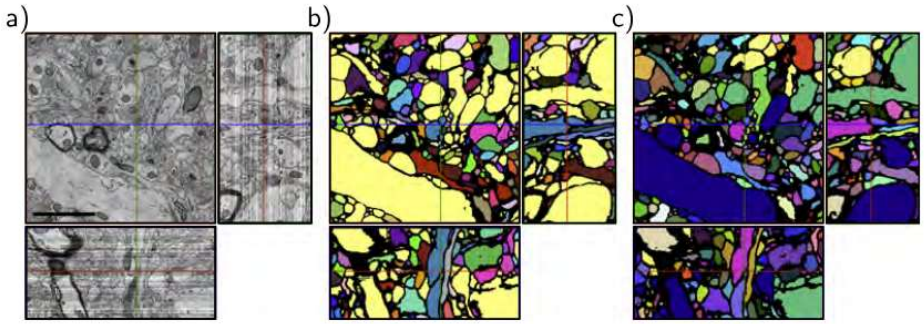
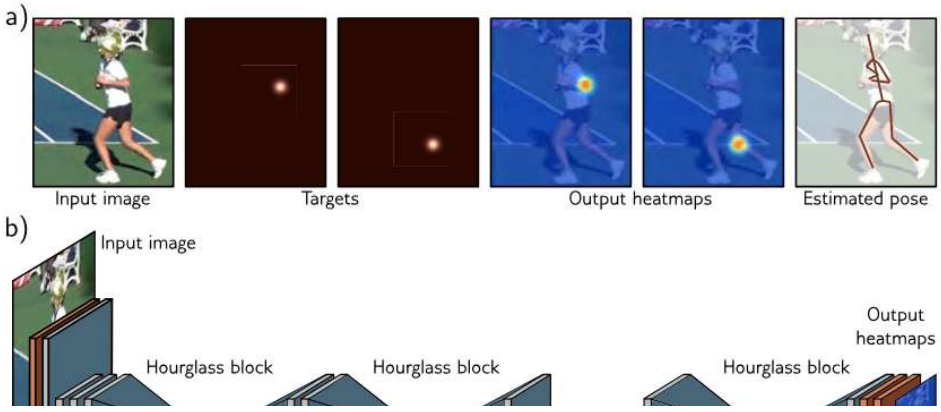


Figure 11.11 Segmentation using U-Net in 3D. a) Three slices through a 3D volume of mouse cortex taken by scanning electron microscope. b) A single U-Net is used to classify voxels as being inside or outside neurites. Connected regions are identified with different colors. c) For a better result, an ensemble of five U-Nets is trained, and a voxel is only classified as belonging to the cell if all five networks agree. Adapted from Falk et al. (2019).



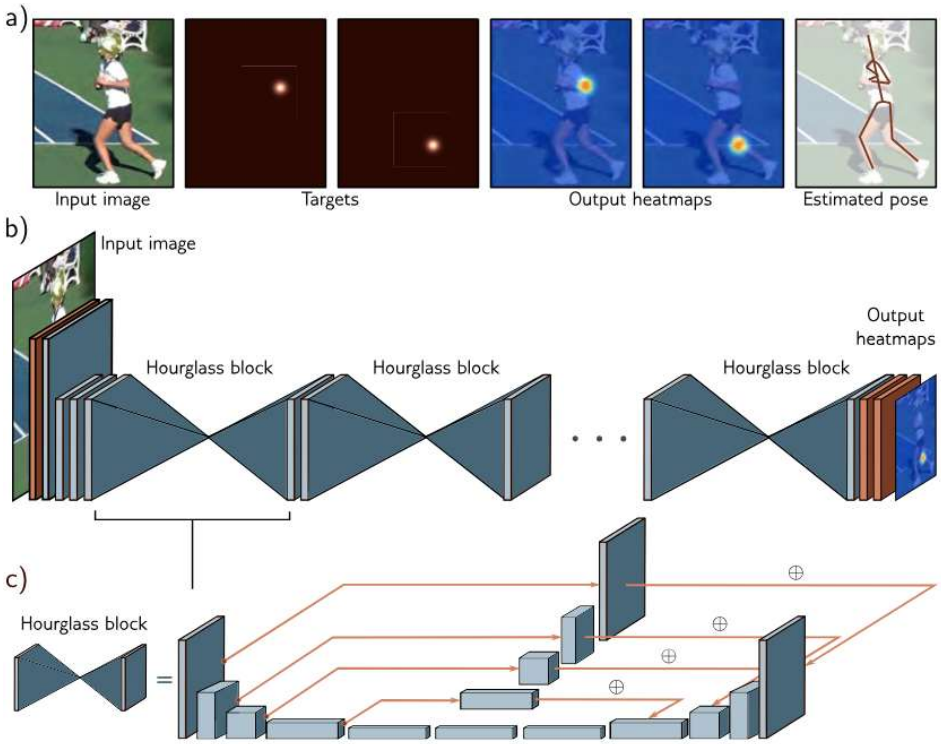


Figure 11.12 Stacked hourglass networks for pose estimation. a) The network input is an image containing a person, and the output is a set of heatmaps, with one heatmap for each joint. This is formulated as a regression problem where the targets are heatmap images with small, highlighted regions at the ground-truth joint positions. The peak of the estimated heatmap is used to establish each final joint position. b) The architecture consists of initial convolutional and residual layers followed by a series of hourglass blocks. c) Each hourglass block consists of an encoder-decoder network similar to the U-Net except that the convolutions use zero-padding, some further processing is done in the residual links, and these links add this processed representation rather than concatenate it. Each blue cuboid is itself a bottleneck residual block (figure 11.7b). Adapted from Newell et al. (2016).

11.6: Why do nets with residual connections perform so well?

- Its possible to extend the ResNet architecture to 1000 layers & still train effectively.
- The improvement in image classification performance was originally attributed to additional network depth, but 2 pieces of evidence contradict this:
  1) Shallower, wider resnets sometimes outperform deeper, narrower ones, with a comparible parameter count.
     ↳ So better performance can be achieved w/ fewer layers but more channels per layer.
  2) There is evidence that the gradients during training don't propagate effectively through very long paths in the unraveled network.
     ↳ So a very deep network may act more like a combination of shallower networks.
- The current view is that residual connections add some value of their own, as well as allowing deeper networks to be trained.
  ↳ Supported by the fact that the loss surfaces of residual networks around a minimum tend to be smoother & more predictable than those for the same network when the skip connections are removed.
  ↳ May make it easier to learn a good solution that generalizes well.

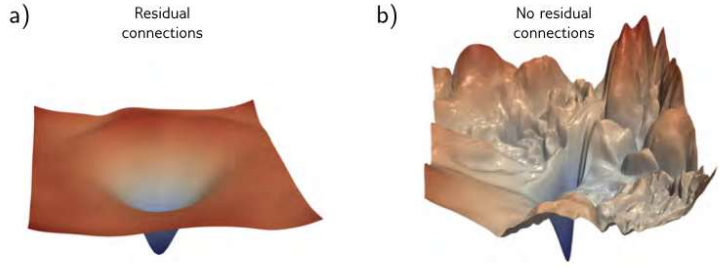


Figure 11.13 Visualizing neural network loss surfaces. Each plot shows the loss surface in two random directions in parameter space around the minimum found by SGD for an image classification task on the CIFAR-10 dataset. These directions are normalized to facilitate side-by-side comparison. a) Residual net with 56 layers. b) Results from the same network without skip connections. The surface is smoother with the skip connections. This facilitates learning and makes the final network performance more robust to minor errors in the parameters, so it will likely generalize better. Adapted from Li et al. (2018b).

11.7: Summary:

- Increasing networking depth indefinitely causes both training & test performance for image classification to decrease.
  - ↳ Could be due to the gradient of the loss w/ respect to parameters early in the network changes quickly & unpredictively relative to the update step size.
- Residual connections add the processed representations back to the input.
  - ↳ Now each layer contributes to the output directly as well as indirectly, so propagating gradients through many layers is not mandatory, & the loss surface is smoother.
- ResNets don't suffer from vanishing gradients but introduce an exponential increase in the variance of the activations during forward propagation & corresponding problems w/ exploding gradients.
  - ↳ Usually handled by adding batch normalization, which compensates for the empirical mean & variance of the batch & then shifts & rescales using learned parameters.
  - ↳ If these parameters are initialized well, very deep networks can be trained.
- There's evidence that both residual links & batch normalization make the loss surface smoother, allowing larger learning rates.
  - ↳ Also, variability in batch statistics adds a source of regularization.
- Residual blocks have been used in convolutional networks, allowing deeper networks to be trained w/ increasing performance.
- Variations of the residual architecture include the DenseNet, which concats the outputs of all prior layers to feed into the current layer, & U-Nets which incorporate residual connections to encoder-decoder models.